최종안

23

ETA 기반 의사결정을 통한 다중 GPU 지식 증류 파이프라인에서의 장애 대응 기법

(ETA-based Failure-Aware Scheduling for Multi-GPU Knowledge Distillation Pipelines)

캡스톤디자인 스타트 연구트랙

23팀 Synapse

2276106 문원정

2276337 최지희

2371023 김현영

지도교수: 심재형 교수님

Table of Contents

1. 팀 정보 3

1. 과제명 3

2. 팀 정보 3

3. 팀 구성원 3

2. 과제 요약 4

1. 문제 정의 4

2. 기존 연구와의 비교 6

3. 제안 내용. 7

4. 기대 효과 및 의의 9

5. 주요 기능 리스트 11

3. 과제 설계 14

1. 요구사항 정의 14

2. 전체 시스템 구성 14

4. 참고 문헌. 15

1. 팀 정보

1.1 과제명

ETA 기반 의사결정을 통한 다중 GPU 지식 증류 파이프라인에서의 장애 대응 기법

(ETA-based Failure-Aware Scheduling for Multi-GPU Knowledge Distillation Pipelines)

1.2 팀 정보

23팀 Synapse

1.3 팀 구성원

1) **2276106 문원정 (프로젝트 총괄 및 시스템 제어)**

- **역할:** 프로젝트 방향성/일정 관리 및 TSPipe 기반 Failover 시스템 총괄
- **설계 & 구현:** ETA 기반 최적 복구 구조 및 실시간 모니터링 아키텍처 설계. EMA 기반 Slowdown 계산 로직, 안전한 체크포인트 저장 및 파이프라인 재시작(Restart) 연동 구현.
- **검증:** 장애 발생 시 시스템 복구 시간(Recovery Time) 측정 및 전체 학습 파이프라인 안정성 검증.

2) **2276337 최지희 (프로파일링 및 성능 모델링)**

- **역할:** KD 파이프라인 프로파일링 데이터 관리 및 수학적 성능 모델링
- **설계 & 구현:** 연산(Compute) 및 통신(Communication) 비용 분해 모델 설계. Base 프로파일링 데이터(CSV) 구축 및 Slowdown Factor가 실시간으로 반영되는 스케일링 로직 구현.
- **검증:** 프로파일링 데이터 정합성 확인 및 Slowdown 주입에 따른 시스템 성능 변화 심층 분석.

3) **2371023 김현영 (동적 파티셔닝 및 알고리즘 개발)**

- **역할:** 동적 프로그래밍(DP) 기반 파이프라인 재분할 및 정책 설계
- **설계 & 구현:** KEEP, REPLAN, DEGRADE 단계적 복구 정책 및 ETA 기반 의사결정 알고리즘 설계. DP 기반 최적 재분할 로직(Optimizer) 재현 및 YAML을 통한 실제 시스템 정책 반영 로직 완성.
- **검증:** 정책별 시스템 중단 시간(Downtime), 복구 오버헤드, 처리량(Throughput) 등 핵심 성능 지표 정량 평가.

2. 과제 요약

2.1 문제 정의

2.1.1 문제 배경

- **다중 GPU 지식 증류(KD) 학습의 보편화:** 최근 거대 AI 모델을 가볍게 만들기 위해, 여러 대의 GPU를 활용하여 크고 무거운 Teacher 모델의 지식을 작은 Student 모델로 전달하는 파이프라인 병렬 처리(Pipeline Parallelism) 기법이 필수적으로 사용되고 있다.
- **KD 파이프라인의 구조적 한계 (유휴 시간/Bubble 발생):** 지식 증류 학습은 Teacher 모델의 순전파(Forward)와 Student 모델의 순/역전파(Forward/Backward)가 순차적으로 맞물려야 하는 복잡한 의존성을 가진다. 이로 인해 파이프라인 구조상 필연적으로 특정 GPU가 연산을 멈추고 데이터를 기다려야만 하는 구조적 유휴 시간(Idle Time / Bubble)이 발생한다.
- **장애 발생 시 유휴 시간의 기하급수적 증가 (Straggler Effect):** 장시간 학습 중 발열이나 하드웨어 결함 등으로 단 하나의 GPU에라도 지연(Slowdown)이 발생하면, 위에서 언급한 복잡한 의존성과 맞물려 정상 GPU들의 대기 시간(Bubble)이 걷잡을 수 없이 커지게 된다. 결국 전체 파이프라인의 처리량이 가장 느린 GPU의 속도에 맞춰지는 심각한 병목 현상이 발생한다.

2.1.2 Target Customer 정의

- 핵심 타겟: 장시간 다중 GPU 기반의 지식 증류(KD) 파이프라인 학습을 수행하는 AI/ML 연구팀 및 엔지니어.

- 세부 타겟: 제한된 GPU 자원 환경에서 반복적인 모델 학습 및 실험을 수행해야 하는 연구자.

2.1.3 핵심 Pain Points

- **정적 파티셔닝의 한계 및 지식 증류(KD) 연산 비대칭성으로 인한 파이프라인 불균형**

  기존 GPipe, PipeDream, TSPipe 등 파이프라인 병렬 학습 프레임워크는 단순히 레이어 수 기준이나 경험적 휴리스틱에 의존하여 모델을 분할(Static Partitioning)한다. 이는 순전파(Forward)만 수행하는 Teacher와 순/역전파(Forward+Backward)를 모두 수행하는 Student 간의 극심한 연산 비용 비대칭성을 전혀 반영하지 못한다. 또한 PCIe 통신 비용, Activation/Gradient 크기 차이 등 하드웨어 특성과 실제 메모리 사용량을 고려하지 않아 특정 Stage가 전체 속도를 제한하는 병목(Bottleneck)으로 작용한다. 결과적으로 CUDA Stall 및 GPU 유휴 시간(Idle Time)이 급증하여, 이론적으로 훌륭한 파이프라인 구조임에도 실제 환경에서는 GPU 자원이 심각하게 낭비되는 것이 첫 번째 핵심 과제이다.

- **잔여 학습 시간(ETA)과 복구 비용을 고려한 수학적 의사결정(Failover) 모델 부재**

  장시간 학습 중 특정 GPU에 예기치 못한 지연(Slowdown)이 발생했을 때, 기존 시스템은 현재 파티션을 유지(KEEP)할지, 오버헤드를 감수하고 재분배(REPLAN)할지, 문제의 GPU를 완전히 배제(DEGRADE)할지 판단할 정량적인 기준이 없다. 최적의 대처 방법은 앞으로 남은 학습 스텝 수와 시스템을 멈추고 재시작하는 데 드는 복구 비용(Restart Cost)에 따라 완전히 달라진다. 따라서 이러한 요소를 종합적으로 계산하여, 각 정책별 최종 예상 완료 시간(ETA)을 수치화하고 가장 빨리 학습을 끝낼 수 있는 최적의 대응 정책을 실시간으로 선택하는 '비용 인지형 예측 기반 동적 파티셔닝' 기술의 부재가 두 번째로 해결해야 할 핵심 과제이다.

2.2 기존 연구와의 비교

1) 범용 파이프라인 병렬 학습 연구: GPipe & PipeDream

- GPipe: 대규모 모델을 레이어 시퀀스로 나누어 여러 가속기에 배치하고, 미니 배치(Mini-batch)를 마이크로 배치(Micro-batch)로 쪼개어 학습하는 범용적인 파이프라인 병렬화 프레임워크를 제안했다. 이를 통해 단일 가속기의 메모리 한계를 뛰어넘었으나, 일반적인 단일 모델 학습을 전제로 하므로 지식 증류(KD) 환경 특유의 비대칭적 결합 구조를 고려하지 않았으며, 학습 중 장애 발생 시의 동적 재구성 기능이 부재한다.
- PipeDream: 연산(Computation)과 통신(Communication)을 중첩시키고, 사전에 측정된 프로파일링(Profiling) 정보를 바탕으로 스테이지(Stage) 간 부하 균형을 시도한 연구입니다. 프로파일링을 파이프라인 구성에 활용했다는 점은 본 연구와 맞닿아 있으나, 역시 범용 DNN 학습을 대상으로 설계되어 KD 맞춤형 스케줄링이나 런타임 장애 복구까지는 다루지 않는다.

2) 가장 직접적인 선행 연구: TSPipe와 그 한계

본 과제와 가장 직접적으로 맞닿아 있는 연구는 TSPipe이다. TSPipe는 Teacher 네트워크에 역전파(Backward)가 없다는 점을 활용하여, Student 연산 사이의 유휴 시간(Bubble)에 Teacher의 순전파(Forward)를 삽입해 GPU 활용률을 비약적으로 높인 훌륭한 스케줄링 기법을 제안했다. 그러나 파이프라인을 '어떻게 나눌 것인가(Partitioning)'와 '중간에 문제가 생기면 어떻게 대처할 것인가(Failover)'에 대해 다음과 같은 치명적인 한계를 가진다.

- 한계 1. 레이어 수 기준의 원시적인 정적 파티셔닝 (Naive Static Partitioning):
  TSPipe는 모델을 분할할 때 특정 레이어의 연산 집중도, 활성화 메모리(Activation memory) 급증 구간, Teacher-Student 간의 연산 비용 비대칭성을 정량적으로 파악하지 못한다. 단순히 '레이어 개수'를 기준으로 모델을 등분하기 때문에 필연적으로 Stage 간 작업 부하 불균형이 발생하고, 파이프라인 유휴 시간 및 CUDA Stall을 유발하는 준최적(Sub-optimal) 상태에 머무르게 된다.
- 한계 2. 런타임 대응력 및 동적 복구 로직 부재:
  정상적인 상태에서의 처리량(Throughput) 극대화에만 초점이 맞춰져 있어, 학습 도중 특정 GPU의 런타임 성능 저하(Slowdown)나 하드웨어 장애(Hard failure)가 발생했을 때 이를 감지하고 스스로 대응 정책을 선택하는 동적 Failover 로직이 전무하다.

3) 본 연구의 포지셔닝 및 핵심 차별성

본 연구는 TSPipe의 효율적인 스케줄링 구조를 계승하면서도, 위에서 언급한 한계점들을 다음과 같은 혁신적인 기법으로 해결한다.

- **프로파일링 기반의 정밀한 동적 파티셔닝**: 사전에 레이어 단위의 실제 연산 시간과 통신 비용을 정밀하게 측정(Profiling)하여, 구조적 비대칭성을 완벽히 반영한 최적의 파티션을 생성한다.
- **ETA 모델 기반의 실시간 런타임 대응 (Failover)**: 학습이 시작된 이후 특정 GPU에 지연이 감지되면, 남은 학습 스텝 수와 복구 오버헤드(Restart Cost)를 수학적으로 수치화한다. 이를 바탕으로 원래 파티션을 유지하는 정책(KEEP), 런타임 도중 파티션을 스스로 다시 짜는 동적 재분할(REPLAN) 및 축소(DEGRADE) 정책을 실시간으로 결정하여 가장 빠른 예상 완료 시간(ETA)을 달성한다.

2.3 제안 내용

본 연구는 기존 TSPipe의 정적 파티셔닝(Static Partitioning)이 가지는 한계를 극복하고, 장시간 다중 GPU 지식 증류(KD) 학습 중 발생하는 성능 저하와 장애 상황에 적응적으로 대응하는 'Failover-Aware Pipeline Training Framework'를 제안한다.

제안하는 시스템은 단순히 파이프라인을 잘 나누는 것을 넘어, 학습 도중 발생하는 동적 환경 변화를 감지하고 가장 빠르게 학습을 완료할 수 있는 최적의 전략을 스스로 찾아내는 End-to-End 복구 시스템이며, 핵심 구성 요소는 다음과 같다.

1) 실시간 성능 모니터링 및 지연(Slowdown) 추정

시스템은 학습 도중 각 GPU와 Stage의 상태를 주기적으로 관찰한다.

- 지표 수집: Step time, Compute time, Communication time 등의 세부 지표를 실시간으로 수집한다.
- EMA 기반 지연 계수 산출: 수집된 정보를 바탕으로 연산 및 통신 측면의 지연 계수(Slowdown Factor)를 추정한다. 일시적인 노이즈에 시스템이 과민 반응하지 않도록 지수 이동 평균(EMA) 방식을 적용하여 실제 지속적인 성능 저하만을 정확히 반영한다.

2) 장애 상황(Failure) 분류 및 맞춤형 대응 정책

런타임 중 발생하는 이상 상황을 증상의 심각도에 따라 두 가지로 분류하고 대응한다.

- Soft Failure (성능 저하): GPU가 완전히 다운되지는 않았으나 발열, 자원 경쟁 등으로 연산/통신이 느려져 파이프라인 전체의 처리량이 감소하는 상황이다. 이 경우 다음 세 가지 정책 후보 중 하나를 평가한다.
    - KEEP (유지): 현재 파티션을 그대로 유지하며 학습을 진행한다.
    - REPLAN (재계획): 전체 GPU 대수는 유지하되, 느려진 상태를 반영하여 파이프라인 Stage를 새롭게 분할(Repartition)한다.
    - DEGRADE (축소): 심각하게 느려진 GPU를 파이프라인에서 완전히 제외하고, 남은 K-1개의 GPU만으로 파티션을 재구성한다.
- Hard Failure (시스템 장애): 특정 GPU가 완전히 다운되어 기존 파이프라인 유지가 불가능한 상황이다. 즉각 학습 루프를 중단하고, 분산 학습 자원(RPC, CUDA 등)을 초기화한 뒤 K-1 체제로 긴급 파티션을 재계산하여 비상 복구 절차를 밟는다.

3) 남은 학습 시간(ETA) 기반의 수학적 의사결정 모델

정책 선택의 핵심 기준은 '전체 학습을 가장 빨리 끝낼 수 있는가'이다. 단순히 당장의 1 Step 속도만 비교하는 것이 아니라, 전체 잔여 학습 시간(ETA)을 계산하여 최적의 대안을 선택한다.

- 비용 종합 계산: 정책 전환에 소요되는 오버헤드(체크포인트 저장/로드, 프로세스 재시작, 파티션 재계산)와 새로운 파티션 적용 후 남은 스텝들을 수행하는 데 걸리는 예상 시간을 모두 합산하여 비교한다.

$$\mathrm{ETA}(p) = C_{\mathrm{restart}}(p) + K_{\mathrm{rem}} \cdot T(p)$$

$$p^* = \arg\min_p \mathrm{ETA}(p)$$

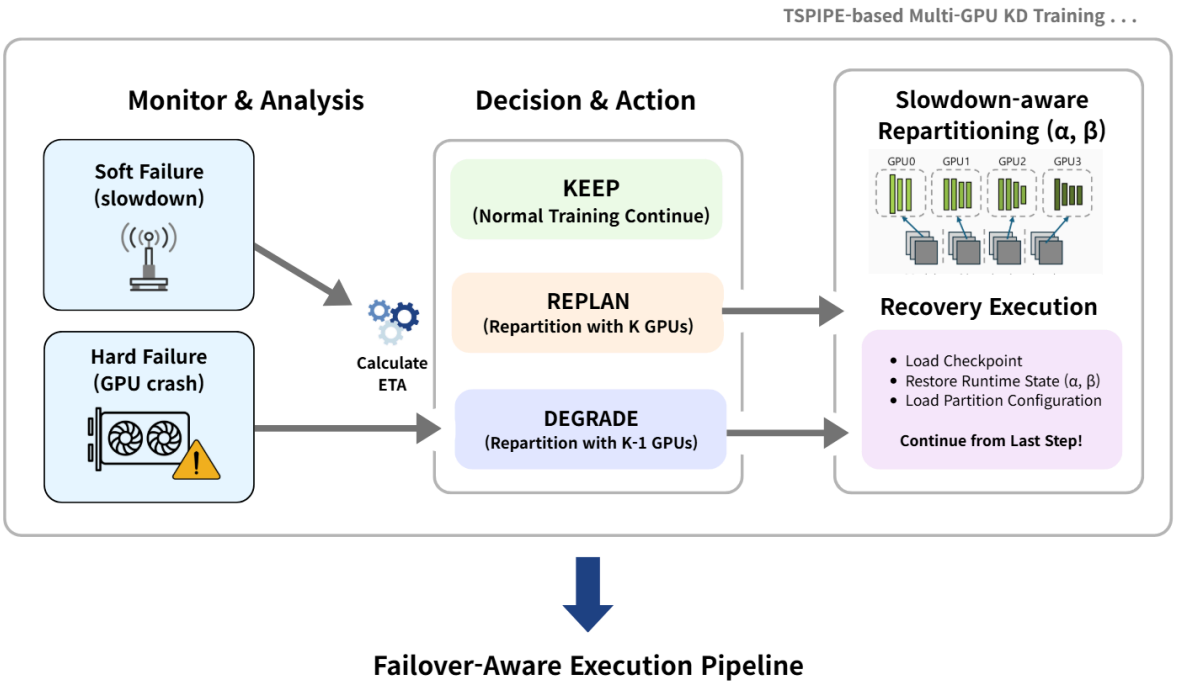
- StageTime 동적 모델링: 파이프라인의 전체 처리량은 '가장 느린 Stage'에 의해 결정된다. 각 Stage의 기본 연산 시간, 출력 통신 시간, 그리고 실시간 관측된 Slowdown 계수를 결합하여 각 정책(KEEP/REPLAN/DEGRADE) 적용 시의 예상 성능을 공정하고 정밀하게 추정한다.

$$\mathrm{StageTime}(g, l, r) = \alpha_{\mathrm{comp}}(g) \sum_{i=l}^{r} t_{\mathrm{comp}}(i) + \beta_{\mathrm{comm}}(g)\, t_{\mathrm{comm_out}}(r)$$

4) 학습 무결성을 보장하는 End-to-End 복구 메커니즘

정책이 결정되면, 학습 문맥(Context)을 잃지 않고 매끄럽게 재시작할 수 있도록 안전한 복구 절차를 수행한다.

- 안전한 중단 및 기록: 정책 실행 직전 Failover Checkpoint를 저장하고, 새롭게 계산된 재분할 정보를 YAML 형태의 Restart Configuration으로 기록한 뒤 프로세스를 종료한다.
- 상태 복원 및 연속 학습: 자동 재시작 시 해당 설정 파일을 우선적으로 읽어 새 파티션을 적용하고, 모델 가중치, Optimizer 상태, Global Step, 그리고 Slowdown 계수까지 모두 복원하여 직전 지점부터 학습을 완벽하게 이어간다.



2.4. 기대 효과 및 의의

본 연구는 단순히 학습 속도를 높이는 것을 넘어, 복잡한 지식 증류(KD) 파이프라인 학습의 '연속성'과 '실전적 신뢰성'을 확보한다는 점에서 다음과 같은 독보적인 의의를 가진다.

1) 지식 증류(KD) 특화형 Failover 프레임워크의 선구적 제시

본 연구는 Teacher와 Student 모델이 동시에 동작하며 복잡한 의존성을 가지는 KD 구조에 최적화된 세계 최초의 Failover-aware 학습 시스템을 지향한다.

- **구조적 비대칭성 극복:** 단일 모델 파이프라인과 달리 KD는 연산 부하의 비대칭성이 극심하여 특정 Stage의 지연이 전체 시스템에 미치는 파급력이 훨씬 크다. 본 연구는 이러한 KD 특유의 구조적 병목을 런타임에 동적으로 해결함으로써, 기존 분산 학습 일반론으로는 도달할 수 없었던 KD 전용 최적화 경로를 제시한다.
- **TSPipe의 신뢰성 중심 확장:** 처리량(Throughput) 중심이었던 기존 TSPipe의 한계를 넘어, 실제 연구 및 서비스 현장에서 필수적인 '안정성'과 '지속성'을 결합하여 기술적 완성도를 한 단계 끌어올렸다.

2) '학습 완료 시간(ETA) 최소화'를 통한 실질적 비용 절감

장애 대응의 초점을 단순히 '복구 여부'가 아닌 '전체 학습 시간의 최적화'에 두었다.

- **수학적 의사결정의 도입:** 지연 발생 시 무조건적인 재시작 대신, 현재 유지(KEEP)와 재구성(REPLAN/DEGRADE)의 비용을 ETA 관점에서 정량적으로 비교한다. 이는 불필요한 체크포인트 로딩 및 오버헤드를 방지하여 전체 학습 완료 시점을 실질적으로 앞당기는 효과를 가져온다.
- **실패 비용(Cost of Failure)의 최소화:** 장시간 소요되는 거대 모델 학습에서 발생할 수 있는 매몰 비용을 최소화하여, 고가의 GPU 자원을 사용하는 연구자와 엔지니어에게 경제적·시간적 이득을 직접적으로 제공한다.

3) 동적 환경에 적응하는 차세대 분산 학습 아키텍처

하드웨어가 항상 완벽하다는 가정을 버리고, '장애를 상수로 전제한 학습 시스템'으로서의 가치를 가진다.

- **자원 활용의 극대화:** 일부 GPU의 성능 저하(Slowdown)가 전체 정상 GPU의 유휴 상태(Idle)로 이어지는 파이프라인의 구조적 취약점을 동적 파티셔닝으로 정면 돌파한다. 이를 통해 어떤 환경에서도 가용한 자원을 100% 활용하는 균형 잡힌 학습을 보장한다.
- **학술적 확장성:** 본 연구에서 제안하는 'ETA 기반 정책 선택'과 '동적 StageTime 모델링'은 향후 다양한 형태의 파이프라인 병렬화 및 거대 언어 모델(LLM) 학습 환경에도 적용 가능한 중요한 학술적 근거가 될 것이다.

2.5 주요 기능 리스트

1) Runtime 모니터링 기능

본 시스템은 학습이 진행되는 동안 각 GPU와 stage의 상태를 지속적으로 관찰하는 기능을 가진다. 이를 위해 step time, compute time, communication time 등의 지표를 주기적으로 수집하며, 이를 통해 현재 파이프라인의 병목 위치와 성능 저하 여부를 파악한다. 이 기능은 이후 slowdown 판단, 정책 선택, 재분할 여부 결정의 기초 입력으로 사용된다는 점에서 전체 시스템의 출발점이 되는 핵심 기능이다.

2) Soft failure 감지 기능

본 시스템은 GPU가 완전히 중단되지 않았더라도 연산 속도 저하나 통신 병목으로 인해 전체 처리량이 감소하는 상황을 soft failure로 정의하고 이를 탐지하는 기능을 포함한다. 단순히 일시적 변동을 감지하는 수준이 아니라, 관측된 성능 변화를 기반으로 현재 slowdown이 지속적인 문제인지 판별하도록 설계되어 있다. 이를 통해 정상 상태와 단순 노이즈, 실제 대응이 필요한 성능 저하 상황을 구분할 수 있다.

3) Hard failure 감지 기능

본 시스템은 특정 GPU에서 실제 장애가 발생하여 기존 파이프라인을 더 이상 유지할 수 없는 상황을 hard failure로 감지하는 기능을 포함한다. GPUHealthMonitor 또는 강제 failure injection을 통해 장애 GPU를 식별하고, 장애 발생 즉시 failover 절차를 시작할 수 있도록 구성되어 있다. 이 기능은 시스템이 장애를 수동으로 확인한 뒤 복구하는 방식이 아니라, 학습 중 자동으로 이상 상황을 인지하고 후속 동작으로 이어질 수 있게 한다는 점에서 중요하다.

4) Slowdown 계수 추정 기능

본 시스템은 각 GPU의 연산 지연과 통신 지연을 정량적으로 반영하기 위해 slowdown 계수를 추정하는 기능을 가진다. 구체적으로는 compute ratio와 communication ratio를 기반으로 alpha, beta 계수를 EMA 방식으로 갱신하며, 이를 통해 순간적인 편차가 아니라 실제 성능 저하 추세를 반영하도록 한다. 이렇게 계산된 계수는 stage time 예측과 repartition 계산, 정책 비교의 핵심 입력으로 사용된다.

5) ETA 기반 정책 선택 기능

본 시스템은 장애 또는 성능 저하 발생 시 KEEP, REPLAN, DEGRADE 중 어떤 대응이 가장 유리한지를 ETA 기준으로 비교하여 선택하는 기능을 가진다. 이때 ETA는 단순 step time만

비교하는 것이 아니라, 체크포인트 저장 시간, 재시작 오버헤드, 체크포인트 로드 시간, 새 partition 적용 이후의 남은 학습 시간까지 포함하여 계산된다. 따라서 이 기능은 단순 복구가 아니라 전체 학습 완료 시간을 최소화하는 방향으로 의사결정을 수행한다.

6) KEEP 기능

KEEP은 현재 partition과 GPU 구성을 그대로 유지한 채 학습을 계속 수행하는 기능이다. 감지된 slowdown이 크지 않거나, 재분할 및 재시작에 필요한 오버헤드가 더 크게 예상되는 경우에 선택된다. 즉 불필요한 재시작을 방지하고, 현재 상태를 유지하는 편이 전체 ETA 관점에서 더 유리할 때 적용되는 기본 대응 기능이다.

7) REPLAN 기능

REPLAN은 현재 사용 중인 전체 GPU는 유지하되, runtime 상태를 반영하여 stage partition을 다시 계산하는 기능이다. 이 기능은 특정 GPU가 상대적으로 느려졌지만 완전히 제외할 정도는 아니라고 판단될 때 유효하다. 기존 profiling 기반의 정적 partition을 그대로 유지하지 않고, 현재 slowdown 계수를 반영하여 새로운 분할 구성을 산출함으로써 전체 파이프라인 병목을 완화하는 역할을 한다.

8) DEGRADE 기능

DEGRADE는 문제가 있는 GPU를 사실상 제외하고 남은 GPU들만으로 K-1 partition을 다시 계산하는 기능이다. 특정 GPU의 slowdown이 매우 크거나 hard failure에 가까운 수준으로 판단되는 경우, 전체 시스템이 해당 GPU에 계속 의존하는 것보다 남은 자원으로 재구성하는 편이 더 유리할 수 있다. 이 기능은 장애 GPU를 제외한 상태에서도 학습을 계속 이어갈 수 있도록 하는 축소 복구 수단이라는 점에서 의미가 있다.

9) Dynamic repartition 기능

본 시스템은 정책 선택 결과에 따라 실제로 새로운 snet partition과 tnet partition을 계산하고 적용하는 기능을 가진다. 이 과정에서 stage time predictor 또는 DP 기반 partition solver를 이용하여 현재 GPU 상태에 적합한 분할점을 산출한다. 이를 통해 초기 고정 분할에 묶이지 않고, runtime 상황 변화에 따라 파이프라인 구조 자체를 재구성할 수 있다.

10) Checkpoint 저장 기능

본 시스템은 failover 상황에서 학습 상태를 보존하기 위해 checkpoint를 저장하는 기능을 포함한다. Soft failure의 경우 failover checkpoint를 저장하고, hard failure의 경우 emergency checkpoint를 저장하여 이후 재시작 시 복구 기준점으로 사용한다. 이 체크포인트에는 student model, teacher model, optimizer state, global step 등 학습 재개에 필요한 정보가 함께 포함된다.

11) Restart configuration 기록 기능

본 시스템은 재시작 시 필요한 partition 정보와 checkpoint 경로, 실패 유형, 사용 가능한 GPU 목록 등을 restart configuration 파일로 저장하는 기능을 가진다. Soft failure와 hard failure는 각각 별도의 restart config를 사용하며, 이를 통해 재시작 과정에서 어떤 유형의 failover가 발생했는지 명확히 구분할 수 있다. 이 기능은 단순 체크포인트 저장을 넘어, 재구성된 학습 환경 전체를 복원하는 데 필요한 메타정보를 함께 관리하는 역할을 한다.

12) Checkpoint 기반 재시작 기능

본 시스템은 저장된 checkpoint와 restart configuration을 바탕으로 학습을 재시작하는 기능을 제공한다. 재시작 시 새 partition을 YAML 설정에 반영하고, teacher/student 모델 상태와 optimizer 상태, global step을 복구하여 마지막 진행 지점부터 학습을 이어간다. 이 기능을 통해 failover 이후에도 처음부터 다시 학습하지 않고, 중단 직전 상태를 최대한 유지하면서 연속적으로 학습을 재개할 수 있다.

13) 분산 환경 정리 및 복구 기능

Hard failure 발생 시 본 시스템은 기존 RPC, process group, CUDA 자원 등을 정리하고, 종료된 worker 및 통신 환경을 정돈한 뒤 새 환경에서 학습을 재시작할 수 있도록 하는 기능을 포함한다. 이는 단순히 프로세스를 껐다 켜는 수준이 아니라, 장애 이후 남은 자원 위에서 다시 정상적인 분산 학습 환경을 형성하기 위한 기능이다. 특히 GPU 장애 직후 통신 상태가 불완전하게 남는 문제를 줄이고, clean restart를 가능하게 한다는 점에서 중요한 기능이다.

14) 학습 연속성 보장 기능

본 시스템의 최종 기능적 목표는 장애나 slowdown이 발생하더라도 전체 학습이 가능한 한 중단 없이 이어지도록 만드는 데 있다. 이를 위해 상태 감지, 정책 선택, checkpoint 저장, repartition, 재시작 복구가 하나의 흐름으로 통합되어 동작한다. 즉 개별 기능들이 따로 존재하는 것이 아니라, 학습 연속성 보장이라는 목적 아래 end-to-end failover framework로 결합되어 있다는 점이 본 시스템의 핵심 기능적 특징이다.

3. 과제 설계

3.1 요구 사항 정리

본 과제의 시스템은 TSPIPE 기반 다중 GPU 지식 증류 학습 환경에서 runtime 중 발생하는 성능 저하와 장애에 대응하여, 학습을 가능한 한 빠르고 안정적으로 지속할 수 있어야 한다. 이를 위해 첫째, 학습 과정에서 GPU별 step time, compute time, communication time을 지속적으로 수집하고 이상 상태를 감지할 수 있어야 한다. 둘째, 감지된 상태를 바탕으로 soft failure와 hard failure를 구분할 수 있어야 하며, soft failure의 경우에는 현재 상태 유지, 전체 GPU 재분할, 일부 GPU 제외 후 재구성 중 적절한 대응 전략을 선택할 수 있어야 한다. 셋째, 각 대응 전략의 효과를 비교하기 위해 slowdown 계수와 stage time을 반영한 ETA를 계산할 수 있어야 한다.

또한 시스템은 정책 선택 이후 실제 복구와 재시작까지 자동으로 수행할 수 있어야 한다. 이를 위해 체크포인트 저장, restart configuration 기록, partition 재적용, 모델 및 optimizer 상태 복구 기능이 필요하다. Hard failure 발생 시에는 장애 GPU를 제외한 K-1 환경에서도 학습을 재개할 수 있어야 하며, soft failure의 경우에는 새 partition을 반영하여 마지막 step부터 연속적으로 학습을 이어갈 수 있어야 한다. 마지막으로 시스템은 기존 TSPIPE 학습 구조와 호환되어야 하며, 추가된 failover 기능이 정상 상황에서 과도한 오버헤드를 유발하지 않도록 설계되어야 한다.

3.2 전체 시스템 구성

본 연구에서 사용하는 실험 환경 및 시스템 구성은 다음과 같다.

구분	내용
Compute Environment	Multi-GPU 시스템 기반 파이프라인 병렬 학습 환경
Hardware	NVIDIA RTX 5060 Ti GPU × 4
Interconnect	PCIe 기반 GPU 간 통신
Operating System	Ubuntu 22.04 LTS (x86_64)

Framework	PyTorch + TSPIPE (Pipeline Parallel Knowledge Distillation Framework)
Model (Teacher)	ViT-Large (forward-only execution)
Model (Student)	ResNet-152 (forward + backward execution)
Training Paradigm	Knowledge Distillation (Teacher-Student 구조)
Dataset	ImageNet-2012
Batch Size	8 ~ 512 (다양한 부하 조건에서 평가)
Profiling Scope	레이어별 연산·메모리 기본 지표와 더불어, 스테이지 간 통신 지연 및 실시간 Compute/Communication Slowdown 보정 계수를 포함한 동적 프로파일링 수행
Optimization Target	구조적 비대칭성과 동적 지연을 고려한 '전체 학습 완료 시간(ETA)의 최소화'
Expected Effect	지연 및 장애 상황에서의 동적 재분할을 통한 누적 학습 비용(Wall-clock Time)의 최소화

4. 참고문헌

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Lim, H. et al. (2022). TSPipe: Learn from Teacher Faster with Pipelines. *Proceedings of the 39th International Conference on Machine Learning*, 162, 13302-13312.

Huang, Y. et al. (2019). GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *Advances in Neural Information Processing Systems*, 32, 103-112.

Narayanan, D. et al. (2019). PipeDream: Generalized Pipeline Parallelism for DNN Training. *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 1-15.

Chen, Y. et al. (2024). FTPipeHD: A Fault-Tolerant Pipeline-Parallel Distributed Training Approach for Heterogeneous Edge Devices. *IEEE Transactions on Mobile Computing*, 23(4), 3200-3212.
Wang, Y. et al. (2023). Fault-Tolerant Hybrid-Parallel Training at Scale with Reliable and Efficient In-memory Checkpointing. *arXiv preprint arXiv:2310.12670*.

필수포함사항

(1)Team Info (팀 정보)

1.1. 과제명 -

1.2. 팀 정보 (팀 번호, 팀 이름 등) -

1.3. 팀 구성원 -

(2) Project-Summary (과제 요약)

2.1. 문제 정의

2.2. 기존연구와의 비교

2.3. 제안 내용

2.4. 기대 효과 및 의의

2.5. 주요 기능 리스트

(3) Project-Design (과제 설계)

3.1. 요구사항 정의

3.2. 전체시스템 구성